

Implementation Plan: CGAL co-refinement + quality remesh for the SAFS 6-fault network

Implementation Plan: CGAL co-refinement + quality remesh for the SAFS 6-fault network

Working directory anchor: `/Users/chunhuizhao/projects/seas-mfem-safs/miniapps/seas/safs/project_7.0_preferred/`. All relative paths below are anchored to that directory unless an absolute path is given.

Overview

`project_7.0_preferred/` produces `data_cleanfreesurf/*_clean_clip.stl` (six SAFS CFM faults × three resolutions, each clipped at $z=0$ and isotropically remeshed by `code_preprocess/clean_freesurface_mesh.py`). Each STL is internally clean — manifold, no $z>0$, well-shaped triangles — but the six STLs **do not share a triangulation along their pairwise intersections**. Eight intersecting pairs exist (documented at [../project_7.0_alternative/code_preprocess/EXPLORE_pymeshlab_intersections.md](#), spatial-overlap table). Without conformal sharing, `gmsh -3` with `Merge + Surface{}` In `Volume{}` produces ill-shaped tets along every fault-fault crossing or fails to close the volume.

This plan builds the same CGAL corefine + Delaunay-remesh pipeline that [../project_7.0_alternative/code_preprocess/PLAN_corefine_pipeline.md](#) prescribes for the single-fault Fuis case, but **specialised to the 6-fault preferred set, run on the already-clipped `_clean_clip.stl` inputs**, and accountable to the user’s two stated quality factors:

- 1. **Mesh size:** every output triangle edge $\geq$ a hard floor (`MIN_EDGE_M`, default 100 m) and $\leq$ a hard ceiling (`MAX_EDGE_M`, default = `target_length`); polyline sub-edges respect both bounds.
- 2. **Triangle/tet quality:** surface triangles satisfy `q_tri = 4√3·A/Σe² ≥ 0.3`; downstream tets satisfy `q_tet ≥ 0.1` (worst) and ≥ 0.3 for $\geq 99.9\%$ (bulk-quality contract from the alternative plan).

Why CGAL is unavoidable: the EXPLORE document [../project_7.0_alternative/code_preprocess/EXPLORE_pymeshlab_intersections.md](#) demonstrates empirically that **pymeshlab cannot produce conformal multi-fault output** — its boolean filter rejects open surfaces, its uniform-resampler destroys the source triangulation, and `meshing_isotropic_explicit_remeshing` is single-mesh by construction. CGAL `PMP::corefine` is the only library function in the conda environment (`cgall-61` ships `cgall-cpp 6.1.1`) that inserts `AnB` exactly into both `A` and `B` while sharing vertex coordinates bit-identically.

This is a planning document. No code is written here. The deliverable is a phased, file-level plan whose acceptance criteria the implementer can verify mechanically.

Inputs and outputs (file-system contract)

```
INPUT (already exists, produced by clean_freesurface_mesh.py):
data_cleanfreesurf/SAFS-SAFZ-COAV-Mission_Creek_fault_strand-CFM4_<R>m_clean_clip.stl
data_cleanfreesurf/SAFS-SAFZ-MJVS-San_Andreas_fault-CFM6_<R>m_clean_clip.stl
data_cleanfreesurf/SAFS-SAFZ-MULT-Southern_San_Andreas_fault_and_Banning-CFM6_<R>m_clean_clip.stl
data_cleanfreesurf/SAFS-SAFZ-SBMT-Garnet_Hill_fault-CFM6_<R>m_clean_clip.stl
data_cleanfreesurf/SAFS-SAFZ-SBMT-Mission_Creek_fault_strand-CFM4_<R>m_clean_clip.stl
data_cleanfreesurf/SAFS-SAFZ-SBMT-San_Andreas_fault-CFM6_<R>m_clean_clip.stl
R ∈ {2000, 1000, 500}; this plan validates R = 2000 only (per locked-in decision Q3).

INTERMEDIATE (created by this plan):
code_preprocess/work/<R>m/*.off          # full-precision OFF intermediates
code_preprocess/work/<R>m/manifest.json  # per-pair / per-fault stats

OUTPUT (created by this plan, consumed by gmsh):
data_corefined/SAFS-SAFZ-*CFM*-<R>m_corefined.stl  # 6 high-precision ASCII STLs
data_corefined/manifest.json                       # listing for the .geo generator
code_meshing/safs_multifault_box-<R>m.geo          # generated multi-fault .geo
```

Naming conventions: - `data_corefined/` is **new**; sits alongside `data_cleanfreesurf/`, `data_preprocess/`, `raw_data/`. Created by Phase 3. - `code_preprocess/corefine_cgal/` is **new** (the C++ tools). Created by Phase 1. - `code_preprocess/work/` is gitignored; created on demand. - `code_meshing/` already exists but is empty; this plan populates it (Phase 4).

Constraints

Hard constraints (each verified by an acceptance criterion)

- **Surface edge floor:** every output `.stl` has `min(edge_length) ≥ MIN_EDGE_M` (default 100 m).

- **Surface edge ceiling:** every output `.stl` has $\max(\text{edge_length}) \leq 1.5 \cdot \text{target_length}$ (the $4/3 \cdot \text{target}$ upper bound from `isotropic_remeshing` semantics, with a small margin).
- **Surface quality:** every output `.stl` has $\min(q_{\text{tri}}) \geq 0.3$ and the *median* $q_{\text{tri}} \geq 0.85$.
- **Pairwise conformality:** for every intersecting pair (A, B) reported by `PMP::intersecting_meshe`s, every vertex on the corefined polyline appears at **bit-identical** coordinates in both meshes when read from OFF, and within $1e-6 \cdot \text{bbox_diag}$ after STL ASCII round-trip with the high-precision writer.
- **Bulk tet quality** (Phase 4): on the merged-and-meshed `.msh`, $\min(q_{\text{tet}}) \geq 0.1$ and $\geq 99.9\%$ of tets have $q_{\text{tet}} \geq 0.3$ per `code_preprocess/msh_to_vtu.py`'s isoperimetric-ratio metric.

Interface constraints

- The downstream consumer is gmsh's `Merge "*.stl"; Surface{tag} In Volume{1}; Mesh.Algorithm3D = 1;` recipe, identical to `../project_7.0_alternative/code_meshing/safs_fault_box_freesurface_clip.geo`. **Do not introduce `BooleanFragments`, `ClassifySurfaces`, or `HXT (Algorithm3D=10)`** — all three were tested and rejected for discrete-STL embedding (see notes in `../project_7.0_alternative/code_meshing/safs_fault_box_buried.geo:L84-L89` and the explanatory header of `safs_fault_box_freesurface_clip.geo:L7-L21`).
- `code_preprocess/ts_to_stl.py`, `code_preprocess/clean_freesurface_mesh.py`, `code_preprocess/msh_to_vtu.py` are **not modified**. The new tooling is additive.

Dependency constraints

- C++ stage is built against the user's `cgall-61` `conda env (cgall-cpp 6.1.1, clang 19.1.7, cmake 4.3, eigen 3.4, boost)`. This env contains no Python interpreter — it is for *compiling* CGAL programs only.
- Python orchestration lives in `code_preprocess/` and runs in `pythonenv` (the env that already runs `clean_freesurface_mesh.py`, `ts_to_stl.py`, `msh_to_vtu.py`). New deps allowed: none beyond what's already present (`numpy`, `meshio`, optionally `pymeshlab`).
- gmsh is invoked from `pythonenv` (where gmsh Python bindings already exist).
- Build artefacts under `code_preprocess/corefine_cgall/build/` are gitignored.

Convention constraints

- C++ source style: match CGAL examples — `using K = CGAL::Exact_predicates_inexact_constructions_kernel;`, `using Mesh = CGAL::Surface_mesh<K::Point_3>;`, `namespace PMP = CGAL::Polygon_mesh_processing;`, `namespace pp = PMP::parameters;`.
- Python style: `argparse` CLI, `main()` $\rightarrow$ `int`, output prints in the same format as `clean_freesurface_mesh.py`.
- Constraint-edge property map idiom: `auto ecm = get(boost::dynamic_edge_property_t<bool>(), mesh);` — same as the alternative plan.

Numerical constraints

- Kernel: `Epick (Exact_predicates_inexact_constructions_kernel)` — exact predicates, double constructions. This matches `igl::copyleft::cgall::mesh_boolean` already linked into `pymeshlab` and avoids precision drift.
- UTM coordinates $\sim 10^6$ m. ASCII I/O must use ≥ 10 significant decimal digits (`%.10g` or 15-digit `printf`); STL conformality is lost if the writer rounds to 6 digits.
- `gmsh.option.setNumber("Geometry.Tolerance", 1e-3)` — explicit, 1 mm; well above the writer precision and well below the smallest expected polyline sub-edge after resampling.

Mathematical content (summary; full statements live inside each phase)

Triangle quality metric (used in Phase 1 / Phase 5 acceptance)

For a triangle with edge lengths `e_a`, `e_b`, `e_c` and area `A`,

$$q_{\text{tri}} = \frac{4\sqrt{3}}{A} \sqrt{e_a^2 + e_b^2 + e_c^2} \in [0, 1], \quad q_{\text{tri}} = 1 \Leftrightarrow \text{equilateral}.$$

Matches `code_preprocess/msh_to_vtu.py`.

Pipeline core (per-pair)

For two open triangulated surfaces `A`, `B` with non-empty intersection: 1. `polylines = PMP::surface_intersection(A, B)` — read-only, exact polyline of `AnB`. 2. `resampled = arc_length_resample(polylines, spacing=POLYLINE_SPACING_M, min_edge=MIN_EDGE_M)` — uniform sub-edges in `[MIN_EDGE_M, 1.5·POLYLINE_SPACING_M]`. 3. `PMP::corefine(A, B, ecm_A, ecm_B)` — `A` and `B` mutated in-place; the constraint maps mark every newly-inserted edge. 4. `A_out = PMP::surface_Delaunay_remeshing(A, polyline_constraints=resampled, protect_constraints=true, mesh_edge_size=MAX_EDGE_M)` — same for `B`; polyline shared

so vertex sequences agree. 5. `PMP::remove_almost_degenerate_faces(A_out, ecm=mark_polyline(resampled), needle_threshold=4, cap_threshold=cos(160°), collapse_length_threshold=1.5·MAX_EDGE_M)` — same for B; closes the slivers that the Delaunay refinement may leave behind.

N-fault generalisation

With `N=6` faults and the 8 known intersecting pairs (overlap table in [../project 7.0 alternative/code_preprocess/EXPLORE pymeshlab intersections.md](#):L75-L83), iterate the per-pair core in **descending order of bbox-overlap volume**, accumulating per-mesh polyline sets. Each mesh is then Delaunay-remeshed *once* with all of its polyline constraints simultaneously. Triple junctions (≥ 3 faults meeting at one polyline) are handled implicitly by the per-pair `surface_intersection`; a Phase 5 sentinel test verifies they did not introduce T-vertices.

The “do every pair, then remesh per mesh” ordering is taken verbatim from [../project 7.0 alternative/code_preprocess/PLAN_corefine_pipeline.md](#):L424-L496, Phase 2. This plan does **not** re-derive that algorithm; it inherits it and tunes parameters for the 6-fault SAFS network.

Phase 1: Build the C++ corefine + repair tooling

Goal

A pair of CLI binaries (`corefine_pair`, `corefine_set`) that take `.off` inputs and emit corefined, Delaunay-remeshed, sliver-repaired `.off` outputs satisfying the hard constraints above. After this phase, on the SBMT-Garnet_Hill × SBMT-Mission_Creek 2000 m pair, a single `corefine_pair` invocation produces two output meshes with `min_edge ≥ 100 m`, `min(q_tri) ≥ 0.3`, polyline conformality verified.

Files to Create

- `code_preprocess/corefine_cgal/CMakeLists.txt` — cmake project, finds `CGAL`, `Boost`, `Eigen3` from `cgal-61`'s `CMAKE_PREFIX_PATH`.
- `code_preprocess/corefine_cgal/corefine_pair.cpp` — single-pair driver. Specification: see “CLI” and “Algorithm” tables below.
- `code_preprocess/corefine_cgal/corefine_set.cpp` — N-fault driver (uses Phase 1 helpers).
- `code_preprocess/corefine_cgal/polyline_resample.h` — header-only, arc-length-uniform polyline resampler. Same interface as the alternative-plan Phase 1 (see [../project 7.0 alternative/code_preprocess/PLAN_corefine_pipeline.md](#):L158-L195).
- `code_preprocess/corefine_cgal/quality_repair.h` — header-only `Stats struct + run / compute_stats / validate` template functions. Same interface as alternative-plan Phase 1 ([PLAN_corefine_pipeline.md](#):L197-L240).
- `code_preprocess/corefine_cgal/intersection_graph.h` — `find_intersecting_pairs(meshes)` and `sort_by_overlap_volume(pairs, meshes)` for the multi-fault driver. Same as alternative-plan Phase 2 ([PLAN_corefine_pipeline.md](#):L498-L513).
- `code_preprocess/corefine_cgal/io_helpers.h` — `read_polygon_mesh_any(path, mesh)`, `write_off(path, mesh)`, `write_stl_high_precision(path, mesh, "%.10g")`. The custom STL writer is required because CGAL's default uses `<<` with default precision (~6 digits) which is **insufficient** for the UTM coordinate magnitudes — empirically loses conformality.
- `code_preprocess/corefine_cgal/.gitignore` — line `build/`.

Files to Modify

- None.

Detailed Requirements

CLI of `corefine_pair`

Flag	Type	Default	Meaning
IN_A (positional)	path	required	first input <code>.off / .stl</code>
IN_B (positional)	path	required	second input
OUT_A (positional)	path	required	first output
OUT_B (positional)	path	required	second output
--mesh-edge-size	double, m	1500.0	MAX_EDGE_M ; passed to <code>surface_Delaunay_remeshing</code>
--min-edge	double, m	100.0	MIN_EDGE_M ; floor checked at <code>quality_repair::validate</code>
--polyline-spacing	double, m	$0.5 \cdot \text{mesh-edge-size}$ (clamped $\geq \text{min-edge}$)	uniform spacing for resampled polyline
--features-angle-bound	double, deg	60.0	passed to <code>surface_Delaunay_remeshing</code>
--max-iterations	unsigned	5	repair-pass iterations
--verbose	flag	off	print per-step stats

CLI of `corefine_set`

Flag	Type	Default	Meaning
IN_DIR (positional)	path	required	directory of input <code>.off</code>
OUT_DIR (positional)	path	required	directory for output <code>.off</code> and <code>manifest.json</code>
--ext	string	<code>.off</code>	input extension to scan
--mesh-edge-size	double, m	1500.0	as above
--min-edge	double, m	100.0	as above
--polyline-spacing	double, m	$0.5 \cdot \text{mesh-edge-size}$	as above
--features-angle-bound	double, deg	60.0	as above
--manifest	path	<code>OUT_DIR/manifest.json</code>	output JSON file
--verbose	flag	off	as above

Algorithm of `corefine_pair::main` (literal, in order)

1. Parse CLI; default `--polyline-spacing = max(0.5*mesh-edge-size, min-edge)`.
2. Mesh A, B; `io_helpers::read_polygon_mesh_any(args.in_a, A)`; same for B. Auto-detects `.off / .stl / .ply / .obj` by extension.
3. **Self-intersection guard** (PMP_corefine precondition): for each of A, B, if `PMP::does_self_intersect(M)` then `PMP::autorefine(M)` and warn. Reason: SAFS data may have near-coincident overlapping triangles at fault perimeters; corefine throws `Self_intersection_exception` if `throw_on_self_intersection=true`, or silently produces malformed output if not.
4. `std::vector<std::vector<Point>> polylines; PMP::surface_intersection(A, B, std::back_inserter(polylines));` If `polylines.empty()`, copy A→OUT_A and B→OUT_B verbatim and exit code 2 (“no intersection”).
5. `auto resampled = polyline_resample::resample_all(polylines, args.polyline_spacing, args.min_edge);` Reject any input polyline with < 2 vertices.
6. `auto ecmA = get(boost::dynamic_edge_property_t<bool>(), A); auto ecmB = ...; and PMP::corefine(A, B, pp::edge_is_constrained_map(ecmA), pp::edge_is_constrained_map(ecmB));`
7. `Mesh A_out = PMP::surface_Delaunay_remeshing<Mesh>(A, pp::polyline_constraints(resampled).protect_constraints(true).mesh_edge_size(args.mesh_edge_size).features_angle_bound(args.features_angle_bound));` and same for B_out. **protect_constraints(true) is correct here** because the polyline has *already been resampled* in step 5 to satisfy the $\leq 4/3 \cdot \text{mesh_edge_size}$ precondition — the trap documented at `PLAN_corefine_pipeline.md:L342-L351` (locking corefine slivers in place) is avoided by the prior resampling.
8. `mark_polyline_as_constrained(A_out, resampled, ecm_out_A, tol=1e-9)` — re-derive an edge-constraint map by matching vertex coordinates against resampled. Same for B.
9. `auto stA = quality_repair::run(A_out, ecm_out_A, args.min_edge, /*q_floor=*/0.3, /*cap_threshold_cos=*/cos(160°), /*needle_threshold=*/4.0, args.max_iterations);` same for B. The function iterates `PMP::remove_almost_degenerate_faces` until convergence or `max-iterations`, then runs `PMP::tangential_relaxation` once for final smoothing.
10. **Conformality check:** every polyline vertex must appear (within 1e-9 m) in both A_out and B_out. If any miss, exit 3 with offender list.
11. **Hard-floor validation:** `quality_repair::validate(A_out, ecm_out_A, args.min_edge, 0.3)` and same for B. If false, exit 4 with first 10 offending edge/face coords.
12. `io_helpers::write_off(args.out_a, A_out);` (and B). If output paths end in `.stl`, `write_stl_high_precision(...)` instead.
13. Print stats block (format below).

Algorithm of `corefine_set::main`

1. Glob `IN_DIR/*<ext>`; load each into `std::vector<Mesh> meshes` and remember basenames.
2. `auto pairs = intersection_graph::find_intersecting_pairs(meshes);` (uses `PMP::intersecting_meshes`).
3. `intersection_graph::sort_by_overlap_volume(pairs, meshes);` — descending bbox-overlap volume.
4. Per-mesh accumulator: `std::vector<std::vector<std::vector<Point>>> all_polylines(meshes.size());`.
5. For each `(i, j)` in sorted order:
 - `polylines = PMP::surface_intersection(meshes[i], meshes[j]);`
 - `resampled = polyline_resample::resample_all(polylines, args.polyline_spacing, args.min_edge);`
 - `PMP::corefine(meshes[i], meshes[j], ecm[i], ecm[j]);`
 - `all_polylines[i].insert(...resampled)` and same for `j`.
6. Per mesh: `output_meshes[i] = PMP::surface_Delaunay_remeshing<Mesh>(meshes[i], pp::polyline_constraints(flatten(all_polylines[i])).protect_constraints(true).mesh_edge_size(args.mesh_edge_size).features_angle_bound(args.features_a`
7. `quality_repair::run(output_meshes[i], ...)` for each `i`.
8. **Pairwise conformality re-check:** for every `(i, j)` in `pairs`, count polyline-vertex matches between `output_meshes[i]` and `output_meshes[j]`. Both counts equal `|resampled[(i,j)]|`. If not, exit 7 with the failing pair list.
9. **Manifest emission** (`OUT_DIR/manifest.json`):

```
{
  "ext": ".off",
  "mesh_edge_size": 1500.0,
  "min_edge": 100.0,
  "polyline_spacing": 750.0,
  "meshes": [
    {
      "input": "<basename>.off",
      "output": "<basename>_corefined.off",
      "n_verts": ..., "n_faces": ...,
      "bbox": { "x_lo": ..., "x_hi": ..., ... },
      "edge_stats": { "min": ..., "median": ..., "max": ... },
      "tri_q_stats": { "min": ..., "p1": ..., "median": ... },
      "intersects_with": [
        { "partner": 1, "polyline_vertex_count": 47, "polyline_length_m": 38600.0 }
      ]
    }
  ],
  "pairs": [
    { "i": 0, "j": 1, "n_polyline_verts": 47, "conformal": true }
  ]
}
```

10. Write each `output_meshes[i]` to `OUT_DIR/<basename>_corefined.off`.

Print-stats format (used by both binaries; matches `clean_freesurface_mesh.py` style)

```
A: V_in=<n> F_in=<n> V_out=<n> F_out=<n>
   edge: min=<m> p1=<m> median=<m> max=<m>
   q_tri: min=< > p1=< > median=< >
B: ... same ...
Polyline: count=<n> resampled_count=<n> min_seg=<m> max_seg=<m>
Conformality: <K>/<K> polyline verts match in both meshes (max diff <m>).
Quality floor passed: yes
```

CMakeLists.txt skeleton

```

cmake_minimum_required(VERSION 3.20)
project(safs_corefine LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
find_package(CGAL REQUIRED)
find_package(Boost REQUIRED)
find_package(Eigen3 REQUIRED NO_MODULE)
include_directories(${CMAKE_CURRENT_SOURCE_DIR})
add_executable(corefine_pair corefine_pair.cpp)
add_executable(corefine_set corefine_set.cpp)
target_link_libraries(corefine_pair PRIVATE CGAL::CGAL Eigen3::Eigen)
target_link_libraries(corefine_set PRIVATE CGAL::CGAL Eigen3::Eigen)
foreach(t corefine_pair corefine_set)
    target_compile_options(${t} PRIVATE -O2 -Wall -Wno-deprecated-copy)
endforeach()

```

Build invocation (used by Phase 5 test harness and by the user locally):

```

conda activate cgal-61
cd code_preprocess/corefine_cgal
mkdir -p build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release && cmake --build . -j 4

```

Interfaces

The C++ binaries expose only their CLI; Phase 3 invokes them via `subprocess.run`. The headers (`polyline_resample.h`, `quality_repair.h`, `intersection_graph.h`, `io_helpers.h`) are private to `code_preprocess/corefine_cgal/` — no other code includes them.

Edge Cases to Handle

- **Empty intersection** (Phase-1 step 4): exit 2; copy inputs verbatim. Justification: MJVS-SAF × SBMT-SAF overlaps only by 0.1×0.1×14.8 km, may not actually intersect; pipeline must not abort the whole batch.
- **Single-point intersection** producing a zero-length polyline: filter at `polyline_resample::resample` (skip polylines with < 2 vertices or total length < `min_edge`).
- **Closed polyline** (a fault loops through another): detect via `first==last` (1e-9 tolerance) and produce a closed resampled polyline. Closing edge length in `[0.5·spacing, 1.5·spacing]`.
- **Coplanar overlap** (two faults sharing a 2-D patch, not a 1-D curve): not expected geologically, but the `surface_intersection` may emit polyline vertices of degree ≥ 3. Detect via degree count; abort with exit 5 and a clear error. **Mitigation:** `cleanfreesurf` isotropic remesher may snap two close faults into coplanar overlap if `target-length` is large; tune `--target-length` upstream.
- **Polyline endpoint lies on a fault perimeter** (the polyline opens onto the free surface or onto a buried fault edge): expected at the SAFS surface trace where intersection lines hit `z=0`. The Delaunay remesher handles this naturally because perimeter edges are constrained-by-default; verify via the conformality check.
- **Self-intersection in input** (Phase-1 step 3): repaired by `PMP::autorefine`, with a logged warning.
- **OFF / STL read failure:** exit 6 with `errno`-style message.
- **Polyline sub-edge < `min_edge` after resampling:** the resampler merges adjacent sub-edges < `min_edge` (`PLAN_corefine_pipeline.md:L172-L177`). Must not produce sub-edges < `min_edge` even at polyline endpoints.

Acceptance Criteria

- ☐ Builds with `conda activate cgal-61 && cd code_preprocess/corefine_cgal && mkdir build && cd build && cmake .. && make`. Zero compiler warnings except `-Wno-deprecated-copy`.
- ☐ `corefine_pair --help` prints all flags above.
- ☐ **Synthetic two-rectangles test** (built into Phase 5; uses two perpendicular 10×10 m rectangles triangulated 4×4 each, intersecting at `y=5,z=0`): both outputs contain a polyline vertex at every `polyline-spacing` interval; vertex coordinates bit-identical between A and B at every polyline vertex; `min_edge ≥ min_edge_arg`.
- ☐ **SBMT-Garnet_Hill × SBMT-Mission_Creek 2000 m smoke test** (largest XY overlap, 38.6×8.1×16.2 km): exit 0; both outputs satisfy `min_edge ≥ 100 m`, `min(q_tri) ≥ 0.3`, `n_edges_below_floor == 0`. Wall-clock < 60 s on the user's laptop.
- ☐ **STL ASCII round-trip preserves conformality:** write outputs as `.stl` via the high-precision writer; read back through `meshio`; for every polyline vertex, `||A_coord - B_coord|| < 1e-6 · bbox_diag`. (Justifies the explicit `Geometry.Tolerance = 1e-3` in Phase 4.)
- ☐ `corefine_set OUT_DIR=...` produces a `manifest.json` that parses with `python -m json.tool` and contains the schema documented above.

Dependencies

- Depends on: `cgal-cpp 6.1.1` in `cgal-61` env (specifically `PMP::corefine`, `surface_intersection`, `surface_Delaunay_remeshing`, `remove_almost_degenerate_faces`, `tangential_relaxation`, `autorefine`, `does_self_intersect`, `intersecting_mesher`).
- Required by: Phase 2 (Python orchestration consumes both binaries), Phase 5 (validation harness).

Phase 2: Python orchestration corefine_faults.py

Goal

A single Python script in `code_preprocess/` that, given a resolution $R \in \{2000, 1000, 500\}$ (this plan validates only 2000), reads the six `data_cleanfreesurf/*_<R>m_clean_clip.stl` files, converts them to `.off` (full-precision), invokes `corefine_set`, and emits the six `data_corefined/*_<R>m_corefined.stl` files plus the `manifest.json`. Wraps build-instruction printing if `corefine_set` is missing.

Files to Create

- `code_preprocess/corefine_faults.py`. Module-level constant `RESOLUTIONS = (2000, 1000, 500)`; CLI default `--res 2000`.

Files to Modify

- `.gitignore` (project root, if it exists at `miniapps/seas/safs/.gitignore`): add `data_corefined/`, `code_preprocess/work/`, `code_preprocess/corefine_cgal/build/`.

Detailed Requirements

CLI

Flag	Type	Default	Meaning
<code>--in-dir</code>	path	<code>data_cleanfreesurf</code>	directory of <code>*_clean_clip.stl</code>
<code>--out-dir</code>	path	<code>data_corefined</code>	output directory
<code>--res</code>	int	2000	resolution suffix to scan; only 2000 is validated by this plan
<code>--mesh-edge-size</code>	float, m	<code>--res</code> (e.g. 2000)	passed through to <code>corefine_set</code>
<code>--min-edge</code>	float, m	100.0	passed through
<code>--polyline-spacing</code>	float, m	<code>0.5*mesh-edge-size</code> (clamped $\geq$ min-edge)	passed through
<code>--cgal-bin</code>	path	<code>corefine_cgal/build/corefine_set</code>	location of the C++ binary
<code>--workdir</code>	path	<code>code_preprocess/work/<R>m</code>	OFF intermediates
<code>--keep-intermediate</code>	flag	off	skip <code>workdir</code> cleanup
<code>--verbose</code>	flag	off	propagated to <code>corefine_set</code>

Algorithm

- Locate inputs: `glob(in_dir / f"*_{res}m_clean_clip.stl")`. If empty, exit 1 with message `no inputs in {in_dir} for res={res}m`.
- Verify there are exactly 6 inputs (at 2000 m the preferred set has six faults). If not, warn and continue.
- Convert each `.stl` to `.off` in `workdir`:
 - `import meshio`; read STL; write OFF with full double precision via `meshio.write(off, mesh, file_format="off")` or a custom writer that does `f"{x:.15g}"` per coordinate (preferred — `meshio`'s OFF writer precision varies by version).
 - Output filename strips `_clean_clip` $\rightarrow$ `<basename>_<R>m.off`.
- Verify `cgal-bin` exists; if not, print:

```
error: corefine_set binary not found at {cgal_bin}
to build: conda activate cgal-61 && cd code_preprocess/corefine_cgal &&
          mkdir build && cd build && cmake .. && make
```

and exit 1.

- `subprocess.run([str(cgal_bin), str(workdir), str(workdir), "--ext", ".off", "--mesh-edge-size", str(mesh_edge_size), "--min-edge", str(min_edge), "--polyline-spacing", str(polyline_spacing), "--manifest", str(workdir / "manifest.json")] + (["--verbose"] if verbose else []), check=True, capture_output=False)`.
- Convert each `*_corefined.off` in `workdir` to ASCII STL with `meshio (file_format="stl-ascii")`; write to `out_dir / f"{base}_corefined.stl"`.

7. Read `manifest.json`; assert every pair has `"conformal": true`. If not, abort with exit 7 and print the offending pair list.
8. Copy `manifest.json` to `out_dir/manifest.json`.
9. If not `--keep-intermediate`: `shutil.rmtree(workdir)`.
10. Print human-readable summary (per-fault and per-pair stats).

Edge Cases to Handle

- **Missing `meshio`** in `pythonenv`: catch `ImportError`, print `pip install meshio` instruction, exit 1.
- **Subprocess non-zero return**: `stderr` from C++ tool already streamed; re-raise via `subprocess.CalledProcessError`.
- **Mesh count $\neq 6$** : not fatal at 2000 m; print warning. (The 1000 m and 500 m sets may add or omit faults; out of scope here.)
- **`out_dir` not empty**: warn but overwrite (predictable behavior; the user re-runs frequently).

Acceptance Criteria

- ☐ On `data_cleanfreesurf/*_2000m_clean_clip.stl` (six inputs): produces `data_corefined/*_2000m_corefined.stl` (six outputs) and `data_corefined/manifest.json`. Wall-clock < 5 minutes on the user's laptop.
- ☐ Each output STL parses with `meshio.read(...)` without warnings; vertex/face counts match the manifest within ± 0 (no silent dedup).
- ☐ Manifest has 8 pairs all with `"conformal": true`.
- ☐ All six outputs satisfy `min(edge)  $\geq$  100 m`, `min(q_tri)  $\geq$  0.3`, `median(q_tri)  $\geq$  0.85` (re-verified in Python from the manifest).
- ☐ Re-running with the same args is idempotent (same number of vertices/faces in each output to within 1 % — modulo any non-determinism in `surface_Delaunay_remeshing`).

Dependencies

- Depends on: Phase 1 (`corefine_set` binary).
- Required by: Phase 4 (`.geo` generator reads `data_corefined/manifest.json` and `*_corefined.stl`); Phase 5 (validation harness invokes this script).

Phase 3: Multi-fault gmsh `.geo` and generator

Goal

A generated `code_meshing/safs_multifault_box_2000m.geo` that mirrors `../project_7.0_alternative/code_meshing/safs_fault_box_freesurface_clip.geo` but merges and embeds **all six** corefined faults inside a single bounding-box volume. Generated by a Python helper that reads `data_corefined/manifest.json` and writes the `.geo`. Verifies that gmsh meshes the geometry with the proven `Surface{} In Volume{}` recipe and produces tet-quality satisfying the bulk constraints.

Files to Create

- `code_meshing/safs_multifault_box_2000m.geo` — generated, multi-fault, free-surface variant. Hard-coded box bounds (from manifest) and hard-coded fault filenames.
- `code_preprocess/generate_multifault_geo.py` — generator that reads the manifest and produces the `.geo`.

Files to Modify

- None.

Detailed Requirements

`.geo` template (literal; the generator instantiates the `<...>` placeholders)


```

// =====
// safs_multifault_box_2000m.geo (generated; do not edit by hand)
//
// Embed the six corefined SAFS faults inside a 3-D bounding box.
// Fault sources: ../data_corefined/*_2000m_corefined.stl, produced by
// code_preprocess/corefine_faults.py (CGAL corefine + Delaunay remesh,
// see document/PLAN_cgall_corefine_multifault.md).
//
// Generated: <YYYY-MM-DD> by generate_multifault_geo.py
// Manifest: ../data_corefined/manifest.json
//
// Generate mesh:
// gmesh -3 safs_multifault_box_2000m.geo -o safs_multifault_box_2000m.msh
//
// All units: meters. Coordinates: UTM.
// =====
SetFactory("OpenCASCADE");

// 1. Domain bounds: union of corefined-fault bboxes + paddings.
// Filled in from manifest by the generator.
xmin_fault = <XMIN>; xmax_fault = <XMAX>;
ymin_fault = <YMIN>; ymax_fault = <YMAX>;
zmin_fault = <ZMIN>; zmax_fault = 0.00; // top trace clipped at z=0

PAD_XY = 50000.0; // 50 km horizontal margin
PAD_TOP = 100.0; // 100 m of rock above the free surface
PAD_BOTTOM = 25000.0; // 25 km of rock below deepest fault vertex

xmin = xmin_fault - PAD_XY;
xmax = xmax_fault + PAD_XY;
ymin = ymin_fault - PAD_XY;
ymax = ymax_fault + PAD_XY;
zmin = zmin_fault - PAD_BOTTOM;
zmax = zmax_fault + PAD_TOP;
dx = xmax - xmin; dy = ymax - ymin; dz = zmax - zmin;

// 2. Mesh size controls (HARD floor: LC_MIN must equal Phase 1 --min-edge).
LC_MIN = 100.0;
LC_NEAR = 1500.0;
LC_FAR = 10000.0;
DIST_INNER = 3000.0;
DIST_OUTER = 40000.0;

Mesh.MeshSizeMin = LC_MIN;
Mesh.MeshSizeMax = LC_FAR;
Mesh.MeshSizeExtendFromBoundary = 0;
Mesh.MeshSizeFromPoints = 0;
Mesh.MeshSizeFromCurvature = 0;
Geometry.Tolerance = 1e-3; // 1 mm; tighter than STL writer

// 3. Box volume. OCC tags 1..6 for box faces; volume tag 1.
Box(1) = {xmin, ymin, zmin, dx, dy, dz};

// 4. Merge each corefined fault.
// Discrete-surface tags 7..6+N_FAULTS in order of merge.
Merge "../data_corefined/<fault_1>_2000m_corefined.stl"; // tag 7
Merge "../data_corefined/<fault_2>_2000m_corefined.stl"; // tag 8
Merge "../data_corefined/<fault_3>_2000m_corefined.stl"; // tag 9
Merge "../data_corefined/<fault_4>_2000m_corefined.stl"; // tag 10
Merge "../data_corefined/<fault_5>_2000m_corefined.stl"; // tag 11
Merge "../data_corefined/<fault_6>_2000m_corefined.stl"; // tag 12

N_FAULTS = 6;
fault_tag_start = 7;
fault_surfs[] = {7, 8, 9, 10, 11, 12};

// 5. Embed each fault. Do NOT call ClassifySurfaces.
For k In {0 : N_FAULTS - 1}
    Surface{fault_surfs[k]} In Volume{1};
EndFor

// 6. Distance-based size field (uses ALL fault surfaces).
Field[1] = Distance;
Field[1].SurfacesList = {fault_surfs[]};
Field[1].Sampling = 100;

Field[2] = Threshold;
Field[2].InField = 1;
Field[2].SizeMin = LC_NEAR;
Field[2].SizeMax = LC_FAR;
Field[2].DistMin = DIST_INNER;
Field[2].DistMax = DIST_OUTER;
Background Field = 2;

// 7. Physical groups.
Physical Volume("rock", 1) = {1};
Physical Surface("fault_COAV_Mission_Creek", 101) = {fault_surfs[0]};
Physical Surface("fault_MJVS_San_Andreas", 102) = {fault_surfs[1]};

```

```
Physical Surface("fault_MULT_S_SAF_Banning",      103) = {fault_surfs[2]};
Physical Surface("fault_SBMT_Garnet_Hill",        104) = {fault_surfs[3]};
Physical Surface("fault_SBMT_Mission_Creek",      105) = {fault_surfs[4]};
Physical Surface("fault_SBMT_San_Andreas",        106) = {fault_surfs[5]};
Physical Surface("top",      200) = {6};
Physical Surface("bottom",  201) = {5};
Physical Surface("sides",   202) = {1, 2, 3, 4};

// 8. Mesh algorithms (proven on single-fault freesurface_clip variant).
Mesh.Algorithm = 6;      // 2-D: Frontal-Delaunay
Mesh.Algorithm3D = 1;    // 3-D: Delaunay (HXT/Algorithm3D=10 fails on
                        // embedded discrete STLs)
```

The generator emits the **canonical fault order** (alphabetical by fault tag string); the per-Physical-Surface IDs (101..106) are stable across runs so MFEM's BC-attribute mappings remain valid. If a fault is removed or added in a future run, the IDs above must be updated by hand or the generator extended to assign IDs from the manifest.

generate_multifault_geo.py requirements

- Reads `data_corefined/manifest.json`.
- Computes the union bbox (`xmin_fault`, `xmax_fault`, ...) by walking `meshes[].bbox`.
- Fills the `<XMIN>`, `<XMAX>`, etc. placeholders.
- Fills the six `<fault_k>` placeholders in the merged-files block, in alphabetical order of basename.
- Writes the `.geo` to `code_meshing/safs_multifault_box-<R>m.geo`.
- CLI: `--manifest`, `--res`, `--out-geo`, `--pad-xy`, `--pad-top`, `--pad-bottom`. Defaults match the template.
- Idempotent: re-running with the same manifest produces a byte-equal `.geo` (so `git diff` is meaningful).

Conformality preservation through STL Merge

gmsh's `Merge` for STL deduplicates vertices within `Geometry.Tolerance`. Default is `1e-8 · bbox_diag`; for a box ~500 km across that's ~5 mm, **insufficient at the writer-precision floor of `%10g`** (which gives sub-mm). The template sets `Geometry.Tolerance = 1e-3` (1 mm) explicitly to make tolerance independent of bbox size. The Phase 1 high-precision STL writer is required for this guarantee.

Verification protocol

After `gmsh -3 safs_multifault_box_2000m.geo -o safs_multifault_box_2000m.msh`: 1. Run `code_preprocess/msh_to_vtu.py` to split into `*_bulk.vtu` + per-fault `*_<fault>.vtu`. (No code change needed; `msh_to_vtu.py` already iterates over physical surfaces.) 2. Run the existing quality analysis (the same code that produced `safs_fault_box_buried_bulk.vtu` quality stats in the alternative project) to verify: - Shortest edge anywhere $\geq LC_MIN = 100\text{ m}$. - Worst tet $q_tet \geq 0.1$. - $\geq 99.9\%$ of tets have $q_tet \geq 0.3$.

Edge Cases to Handle

- **Disjoint fault subsets** (intersection graph has multiple components): no special handling — each component meshes as a set of constraints inside the same volume.
- **Fault touching box boundary**: with `PAD_XY = 50 km` and `PAD_BOTTOM = 25 km`, no fault should reach a side or bottom face. If the bbox dimensions in the manifest say otherwise, the generator must fail loudly with `assert pad_top > 0` and `zmax_fault + pad_top > zmax_fault`. (Top edge at `z=0` is fine — we use the `freesurface` convention.)
- **Triple-junction at z=0** (three faults meeting on the free surface): handled by the Phase 1/2 triple-junction sentinel test (Phase 5 Test 4b). If sentinel passes, gmsh sees a degree-3 polyline vertex on the box top face which is a topologically valid configuration.
- **Manifest missing required fields**: generator validates schema; exits 1 with the missing field listed.

Acceptance Criteria

- ☐ `python code_preprocess/generate_multifault_geo.py --res 2000` produces a `code_meshing/safs_multifault_box_2000m.geo` whose MD5 is stable across re-runs.
- ☐ `gmsh -3 safs_multifault_box_2000m.geo -o /tmp/x.msh` exits 0; gmsh log reports 0 ill-shaped tets.
- ☐ The generated `.msh` parses via `meshio`, has 1 volume + 6 fault surface physical groups (IDs 101..106) + 3 boundary groups (IDs 200..202).
- ☐ Bulk tet quality: $\min q_tet \geq 0.1$, $\geq 99.9\% q_tet \geq 0.3$, shortest edge $\geq 100\text{ m}$.
- ☐ Each fault Physical Surface has the same triangle count as the corresponding `_corefined.stl` — gmsh preserved the triangulation as a 2-D constraint without re-meshing it (the proven behavior of `Surface{ } In Volume{ }` with `Algorithm = 6`, `Algorithm3D = 1`).
- ☐ Wall-clock for `gmsh -3` on the 2000 m geometry < 30 minutes on the user's laptop. (Time budget; not a correctness criterion.)

Dependencies

- Depends on: Phase 2 (`data_corefined/*_corefined.stl` and `manifest.json`).
- Required by: nothing in this plan; downstream MFEM consumes the resulting `.msh` via the existing BP5/SAFS driver (out of scope).

Phase 4: Sensitivity sweep over the two factors

Goal

A short, automated parameter sweep that quantifies how output triangle/tet quality and mesh count depend on `mesh-edge-size` and `min-edge`. The deliverable is a markdown report with per-configuration stats; no code change to Phases 1–3 is required.

This phase exists because the user’s two stated factors (mesh size, low triangle/tet quality) are the levers the implementer will tune. We need numbers, not assertions, to defend the default choices in the plan.

Files to Create

- `code_preprocess/sensitivity_sweep.py` — driver that loops over a small grid and runs Phases 2+3 each time into a temp dir.
- `document/REPORT_sensitivity_<DATE>.md` — emitted by the driver; generated, not hand-written.

Files to Modify

- None.

Detailed Requirements

Sweep grid (small; 4 configs)

Config	mesh-edge-size	min-edge	polyline-spacing	Purpose
baseline	1500	100	750	the plan’s default
coarser	2000	100	1000	matches the cleanfreesurf _2000m resolution; mesh-count baseline
finer	1000	100	500	doubles polyline density; should improve worst tet
tighter_floor	1500	200	750	tests whether raising min-edge to 200 m loses pairwise conformality on triple junctions

Total: 4 runs of `corefine_faults.py --res 2000` and 4 runs of `gmsh -3 ...`. Estimated wall-clock 30–60 minutes.

Per-config metrics (extracted from manifest + msh)

- Output triangle count per fault.
- `min(edge_length)` and `min(q_tri)` per fault.
- Bulk tet count.
- `min(q_tet)`, `p1(q_tet)`, fraction `q_tet ≥ 0.3`.
- Wall-clock breakdown: corefine, remesh, gmsh.

Report layout

```
# Sensitivity sweep — CGAL corefine + gmsh — SAFS 2000 m
Date: 2026-MM-DD

## Defaults defended by this sweep
- mesh-edge-size: 1500 m (baseline outperforms coarser on min q_tet by ...)
- min-edge:      100 m (tighter_floor reduces min q_tri by ... but doubles polyline merges)
- polyline-spacing: 750 m (= 0.5 × mesh-edge-size; finer adds ... verts for negligible quality gain)

## Per-config table
| Config | min q_tri | median q_tri | min q_tet | % q_tet ≥ 0.3 | n_tets |
|---|---|---|---|---|---|
| baseline | ... | ... | ... | ... | ... |
| coarser | ... | ... | ... | ... | ... |
| finer | ... | ... | ... | ... | ... |
| tighter_floor | ... | ... | ... | ... | ... |
```

Edge Cases to Handle

- A config that fails (`corefine_set` exit non-zero, or gmsh failure) is recorded as “FAILED” in the report with the captured stderr; the sweep continues.

Acceptance Criteria

- ☐ All 4 configs run end-to-end. Failures in one config do not abort the others.
- ☐ Report exists at `document/REPORT_sensitivity_<DATE>.md` and is human-readable.
- ☐ At least one config (the baseline) satisfies the hard quality contract ($\min q_{\text{tri}} \geq 0.3$, $\min q_{\text{tet}} \geq 0.1$, $\geq 99.9\% q_{\text{tet}} \geq 0.3$).
- ☐ If the baseline fails, the report explicitly recommends a different default.

Dependencies

- Depends on: Phases 1, 2, 3.
- Required by: nothing.

Phase 5: Validation harness

Goal

End-to-end automated test that exercises Phases 1–3 on the SAFS 6-fault 2000 m fixture, asserts every hard constraint, and emits a reproducibility log. The triple-junction sentinel test is a hard gate.

Files to Create

- `code_preprocess/test_corefine_pipeline.py` — pytest end-to-end test.
- `document/VALIDATION_corefine_2000m_<DATE>.md` — generated validation report (per-fault stats, per-pair conformality matrix, per-triplet sentinel results).

Files to Modify

- None.

Detailed Requirements

Test functions (pytest)

1. `test_build_cgal_binaries`: `build_corefine_pair`, `corefine_set`. `pytest.skip` if `cgal-61` is not active.
2. `test_corefine_synthetic_pair`: Phase 1 synthetic acceptance — two perpendicular rectangles; assert polyline conformality bit-identical.
3. `test_corefine_set_smoke_2000m`: `run_corefine_faults.py --res 2000` into a temp dir; assert exit 0 and 6 outputs.
4. `test_per_fault_quality`: load each `_corefined.stl`; assert $\min(\text{edge}) \geq 100\text{ m}$, $\min(q_{\text{tri}}) \geq 0.3$, $\text{median}(q_{\text{tri}}) \geq 0.85$.
5. `test_pairwise_conformality`: for every pair in `manifest.pairs`, load both meshes; for every polyline vertex, assert it appears in both meshes within $1e-6 \cdot \text{bbox_diag}$.
6. `test_triple_junction_sentinel` (**hard gate, per locked-in decision Q2 in alternative-plan PLAN_corefine_pipeline.md:L965-L975**): for every triplet `(A, B, C)` such that all three pairwise corefines are non-empty, locate the polyline endpoints shared between pairs `(A,B)` and `(A,C)` in mesh A's polyline-vertex set; assert each such endpoint also exists at identical coords (within $1e-6\text{ m}$) in B and in C. If any triplet fails, **the test fails** and the report enumerates the failing triplets so a follow-up plan can address them via “snap pre-extracted triple junctions” or “single autorefine on the union”.
7. `test_gmsh_3d`: `run_gmsh -3 safs_multifault_box_2000m.geo`; parse the `.msh`; assert the bulk-tet criteria. Skip if `gmsh` is not on PATH.
8. `test_emit_report`: write the validation report.

Report layout

```

# CGAL multi-fault corefine - Validation report - 2000 m fixture
Date: 2026-MM-DD
Inputs: data_cleanfreesurf/*_2000m_clean_clip.stl (6 faults)
Outputs: data_corefined/*_2000m_corefined.stl

## Per-fault quality
| Fault | n_verts | n_faces | min q_tri | median q_tri | min edge (m) |
|---|---|---|---|---|
| <fault_1> | ... | ... | ... | ... | ... |
| ... |

## Pairwise conformality (8 pairs expected)
| Pair (i,j) | n polyline verts | conformal? | max diff (m) |
|---|---|---|
| ... |

## Triple-junction sentinel
| Triplet (i,j,k) | polyline endpoints checked | passed? |
|---|---|
| ... |

## Bulk tet quality (gmsh)
- Total tets: ...
- min q_tet: ...
- ≥ 0.3 frac: ...
- shortest edge: ...

## Timing
- corefine pairs: ... s
- per-fault remesh: ... s
- gmsh -3: ... s

```

Edge Cases to Handle

- `cgal-61` env not active: skip Phase 1 build test; the rest of the suite reuses pre-built binaries if `corefine_set` is on PATH.
- `gmsh` not installed: skip `test_gmsh_3d`; the rest of the report is still emitted.
- Failed test: the report still emits, with the failing rows highlighted; pytest fails non-zero.

Acceptance Criteria

- ☐ `pytest code_preprocess/test_corefine_pipeline.py -v` passes on the 2000 m fixture.
- ☐ Total wall-clock < 10 minutes (allowing the gmsh test to dominate).
- ☐ Validation report exists at `document/VALIDATION_corefine_2000m_<DATE>.md`.
- ☐ Triple-junction sentinel passes on all detected triplets, OR the report explicitly enumerates failing triplets and links to a follow-up plan stub.

Dependencies

- Depends on: Phases 1, 2, 3.
- Required by: nothing.

Testing Strategy

Unit tests

- `polyline_resample::resample`: synthetic open and closed polylines; assert (a) first/last preserved, (b) all sub-segments in `[0.5, 1.5] · spacing`, (c) no sub-edge < `min_edge`, (d) total length preserved within `spacing`.
- `quality_repair::compute_stats`: hand-crafted mesh with known min edge, `q_tri`, `n_below_floor`; verify exact match.
- `intersection_graph::sort_by_overlap_volume`: synthetic 4-mesh case; assert deterministic order on tied volumes.

Integration tests (Phase 1)

- Two perpendicular rectangles (synthetic, exact analytical intersection).
- Two slightly-misaligned planes (1° dihedral): `features-angle-bound = 60^\circ` should treat the intersection as a feature.
- Two near-coplanar planes (0.1°): expected to abort with the coplanar-overlap warning (Phase 1 edge case).

End-to-end tests (Phase 5)

- 6-fault SAFS 2000 m: covered by Phase 5 acceptance criteria. Hard gate.
- 6-fault SAFS 1000 m, 500 m: explicitly **out of scope** for this plan (locked-in decision Q3 in alternative plan). A follow-up plan adds these once 2000 m is green.

Reference solutions

- For a single corefine pair, the analytic intersection line is known when the inputs are planes; use as ground truth.
- For multi-fault, no analytic reference; cross-check by:
 1. Total polyline-vertex count in mesh A across all its intersection partners equals the union (no double-counting).
 2. Euler characteristic $V - E + F$ of each output mesh matches a manifold-with-boundary expectation.
 3. STL ASCII round-trip $\rightarrow$ re-read $\rightarrow$ re-compute manifest is bit-stable to within the documented tolerance.

Risk Assessment

High risk

- **surface_Delaunay_remeshing instability on near-coplanar inputs.** The Mesh_3 surface remesher uses 3-D Delaunay refinement internally; near-coplanar surfaces produce flat tetrahedra that may cause non-termination. Mitigation: detect via Phase 1 coplanar edge case (exit 5); reject inputs with dihedral $< \sim 5^\circ$. The SAFS faults dip 60–90°; near-coplanar should not occur in practice but cleanfreesurf's snap-strip *can* create coplanar overlap if the user re-tunes `--target-length`.
- **STL precision-induced conformality loss.** Default ASCII STL writers use ~ 6 -digit precision; UTM coordinates ($\sim 10^8$ m) need ≥ 10 digits or polyline vertices drift by mm at the writer step, which gmsh's default `Geometry.Tolerance` will then dedup as different vertices. Mitigation: custom `%.10g` writer in `io_helpers.h` (Phase 1) plus explicit `Geometry.Tolerance = 1e-3` in the `.geo` (Phase 4). Acceptance criterion 4 of Phase 1 verifies this empirically.
- **Multi-pair corefine ordering.** `corefine(A, B)` modifies A in-place; subsequent `corefine(A, C)` sees the modified A. If the (A,B) corefine introduces vertices very close to ANC , exact predicates may produce a slightly different polyline than expected. Mitigation: process pairs in deterministic descending-bbox-overlap-volume order (Phase 1, `intersection_graph::sort_by_overlap_volume`); document that re-ordering may give bit-different but equally valid output.

Medium risk

- **PMP::corefine self-intersection precondition.** Inputs with near-coincident faces (which SAFS data may exhibit at fault perimeters where two faults touch the surface trace) can cause `corefine` to throw. Mitigation: Phase 1 step 3 pre-cleans each input with `PMP::autorefine` if `PMP::does_self_intersect` returns true.
- **Triple junctions via per-pair processing.** CGAL's `surface_intersection` precondition requires “all vertices but endpoints being of degree 2” *per pair*. In a triple junction, mesh A sees a degree-3 polyline vertex when its two incoming intersection lines (with B and C) cross. Mitigation: per-pair processing handles this naturally because each pair's polyline is degree-2; the union of polylines in mesh A may have degree-3 *junction vertices*, but `surface_Delaunay_remeshing` accepts a polyline *set* and treats it as a constraint graph. The Phase 5 sentinel test (`test_triple_junction_sentinel`) is the hard gate.
- **surface_Delaunay_remeshing polyline-resampling determinism.** Two calls with identical `polyline_constraints` *should* produce identical resampled vertex sequences, but this is not explicitly documented. Mitigation: pre-resample once at Phase 1 step 5 and pass already-uniform polylines; the remesher's internal resampling becomes a no-op.
- **gmsh Geometry.Tolerance mismatch with bbox diagonal.** Default is `bbox-diag-relative`; if the user shrinks the box, tolerance scales but STL precision does not. Mitigation: Phase 4 sets `Geometry.Tolerance = 1e-3` explicitly.

Low risk

- **C++ build reproducibility:** `cgal-61` env is pinned; `cgal-cpp` 6.1.1 API stable.
- **Triangulation density blowup at `--res 500`:** not in scope here; flagged for the follow-up plan.

Resolved questions (locked in for this plan)

These are inherited from the alternative plan ([../project 7.0 alternative/code preprocess/PLAN corefine pipeline.md](#):L962-L977) and re-confirmed for the multi-fault preferred case.

- **Q1 — Free-surface clipping is OUT OF SCOPE for this plan.** Inputs are `data_cleanfreesurf/*_clean_clip.stl`, already produced by the upstream `clean_freesurface_mesh.py`. This pipeline does not re-clip, re-snap, or re-remesh the free surface. Top edge sits at $z=0$ by upstream construction.
- **Q2 — Triple junctions: pure pairwise corefine.** Per-pair `surface_intersection + corefine`, accumulate polyline sets, single `surface_Delaunay_remeshing` per mesh. Phase 5 `test_triple_junction_sentinel` is the hard gate. If it fails on the SAFS 2000 m fixture, escalate via a follow-up plan that implements the alternative-plan options (ii) “snap pre-extracted triple junctions” or (iii) “single autorefine on the union”.
- **Q3 — Validation scope: 2000 m only.** Phase 5 fixture is `*_2000m_clean_clip.stl` $\times$ 6 faults. 1000 m and 500 m fixtures are deferred to a follow-up plan. The driver and binaries accept `--res 1000` and `--res 500` but those resolutions are not in the acceptance contract here.

Open questions (NOT resolved; flagged for follow-up)

- **Should the sensitivity sweep (Phase 4) be promoted to a hard gate?** Currently it is descriptive (defends the defaults). If a reviewer disagrees with the defaults, the sweep is the place to argue. No conditional decision tree is built into the plan; out of scope.
- **Does CGAL 6.1.1 have a multi-mesh corefine** (e.g., `corefine_with_constrained_edges` or an `autorefine`-style N-mesh variant)? `PLAN_corefine_pipeline.md`:L437-L449 flags this as worth verifying. If yes, the per-pair loop in Phase 1 (`corefine_set`) could be replaced by a single call. **Action:** Phase 1 implementer searches `corefinement.h` of `cgal-cpp 6.1.1`; if found, file an addendum to this plan before implementing the per-pair loop. If not found, proceed with per-pair as specified.
- **Does `data_corefined/` belong under version control?** The corefined STLs are derived data and reproducible, but they are slow to regenerate (~5 min for 2000 m). Recommendation: gitignore them and keep `manifest.json` in source control as a lightweight provenance file. Out of scope; defer to user.

Defence — why this plan vs. alternatives

- **Why not pymeshlab/MeshLab only?** `EXPLORE_pymeshlab_intersections.md` proves empirically that pymeshlab cannot produce conformal multi-fault output; its boolean filter requires closed solids, its uniform-resampler destroys the source triangulation, its merge filter only welds at exact coordinate coincidence. **No combination of installed pymeshlab filters yields conformal $A \cap B$ for open meshes** (`EXPLORE_pymeshlab_intersections.md`:L165-L167).
- **Why not a single-binary `PMP::autorefine` on the polygon-soup union?** Possible but loses per-fault identity, which Phase 4 needs to assign Physical Surface tags. Per-pair `corefine` preserves the input mesh structure and lets us assign Physical Surfaces deterministically.
- **Why not gmsh's `BooleanFragments` ?** Tested in the alternative project on a single-fault, single-box geometry (notes embedded in `safs_fault_box_buried.geo`:L84-L89 and `safs_fault_box_freesurface_clip.geo`:L7-L21); produces “No tetrahedra in region 1” or “No closed volume” because OCC fragmentation does not work on **discrete (STL-imported)** surfaces — it requires parametric OCC surfaces. Phase 4 of the alternative plan documents an optional retest on the corefined output (`PLAN_corefine_pipeline.md`:L786-L825). This plan deliberately does not include that retest because the SAFS 6-fault network has 8 intersection lines and the BooleanFragments retest scales poorly. If a future reviewer wants the retest documented, it can be added as a Phase 4.5 mirroring the alternative plan.
- **Why CGAL 6.1.1 specifically?** `surface_Delaunay_remeshing` was added in 6.0; `Adaptive_sizing_field` and `relax_constraints` are 5.4+ and 6.0+; the user's `cgal-61` env ships exactly 6.1.1. `EXPLORE_cgal_corefine.md`:L382-L389 enumerates the version-specific features.
- **Why `protect_constraints=true` in `surface_Delaunay_remeshing` rather than `false` ?** Because the polyline is *already resampled* to satisfy the $\leq 4/3 \cdot \text{mesh_edge_size}$ precondition; `true` then preserves the polyline exactly (highest quality), whereas `false` would re-collapse the resampled polyline (sliver-recovery, but at the cost of polyline drift between A and B). The trap (`PLAN_corefine_pipeline.md`:L342-L351) is “raw corefine output as the constraint with `protect_constraints=true`”, which we avoid by resampling first.

Next-step action items if the plan is approved

1. Phase 1 (build C++ tools): ~1 day. Bulk of the work is the cmake configuration and the high-precision STL writer; algorithmic logic is borrowed from the alternative plan.
2. Phase 2 (Python orchestration): ~half day.
3. Phase 3 (gmsh `.geo` generator + smoke test): ~half day.
4. Phase 4 (sensitivity sweep): ~half day; mostly running and tabulating.
5. Phase 5 (validation harness + report): ~half day.

Total estimate: 3 working days for a clean implementation including the report. If Phase 5's triple-junction sentinel fails, add 1–2 days for the follow-up plan that implements alternative-plan option (ii) or (iii).